

Pontifícia Universidade Católica de Goiás

Escola Politécnica e de Artes

Ciência da Computação

Algoritmos em Grafos

Busca em Largura (BFS) e Busca em Profundidade (DFS)

Aluno: Matheus Silva Pains

Matrícula: 2024.1.0028.0032-0

Professor: Dr. Robson Medrado de Oliveira

Disciplina: Algoritmos em Grafos

Prazo: 29/05/2026

Goiânia, 2026

Contents

1	Introdução	2
2	Motivação: por que buscar em grafos?	2
3	Algoritmo genérico de busca	2
4	Busca em Largura (BFS)	3
4.1	Descrição do algoritmo	3
4.2	Implementação	3
4.3	Resultados	6
5	Busca em Profundidade (DFS)	7
5.1	Descrição do algoritmo	7
5.2	Implementação	8
5.3	Resultados	11
6	Comparação entre BFS e DFS	12
7	Conclusão	13

1 Introdução

Este relatório apresenta a implementação, em linguagem C, dos algoritmos de Busca em Largura (*Breadth-First Search* – BFS) e de Busca em Profundidade (*Depth-First Search* – DFS) aplicados a grafos não direcionados.

O desenvolvimento utilizou o ambiente Visual Studio Code como editor e teve como referência teórica o material disponibilizado pelo Prof. Dr. Robson Medrado de Oliveira nas Aulas 10 e 11 da disciplina. Modelos de linguagem natural foram utilizados como apoio para revisão e orientação da implementação.

2 Motivação: por que buscar em grafos?

Grafos são um dos modelos de dados mais versáteis da ciência da computação, permitindo representar relações entre entidades em problemas como redes de computadores, mapas, labirintos e redes sociais. A pergunta central que motiva os algoritmos de busca pode ser formulada da seguinte forma:

Dado um grafo G e um vértice inicial s , quais vértices podem ser alcançados a partir de s ?

Entre as aplicações práticas citadas no material de referência estão: componentes conexas, detecção de ciclos, cálculo de distâncias entre vértices, bipartição, identificação de arestas de corte e resolução de labirintos.

É importante distinguir grafos de árvores: enquanto árvores são grafos sem ciclos e sem arestas paralelas — admitindo percursos naturais como pré-ordem, pós-ordem e por níveis —, grafos gerais podem conter ciclos e múltiplos caminhos entre dois vértices, o que torna necessária uma abordagem sistemática de visitação.

3 Algoritmo genérico de busca

Ambas as implementações deste trabalho derivam de um mesmo algoritmo genérico de busca. A ideia central consiste em construir, passo a passo, uma **árvore de busca** T dentro de G a partir de um vértice inicial s .

A regra de expansão é: enquanto existir uma aresta $xy \in E(G)$ tal que $x \in V(T)$ e $y \notin V(T)$, adicionam-se y e a aresta xy à árvore. O algoritmo termina quando não há mais arestas desse tipo. Se ao final $V(T) = V(G)$, o grafo é conexo; caso contrário, apenas a componente de s foi alcançada.

Conforme destacado no material de referência, *a árvore de busca pode não ser única*, pois depende da ordem de escolha das arestas em cada iteração. O que diferencia

BFS de DFS é exatamente qual aresta (ou qual vértice) é escolhido a cada passo — e, conseqüentemente, qual estrutura de dados controla essa escolha.

4 Busca em Largura (BFS)

4.1 Descrição do algoritmo

A BFS explora o grafo **por níveis**: primeiro visita todos os vizinhos diretos de s , depois os vértices a distância 2, depois os a distância 3, e assim por diante. O critério de escolha é sempre o vértice que foi alcançado *há mais tempo* entre os que ainda possuem vizinhos não visitados.

A estrutura de dados correspondente é a **fila** (FIFO — *First In, First Out*).

O pseudocódigo seguido na implementação é o das Aulas 10 e 11:

```
BuscaLargura(G, s)
  s.visitado = 1
  cria fila vazia F
  Enfileira(F, s)
  enquanto F.tamanho > 0 faça
    u = Desenfileira(F)
    para todo v em N(u) faça
      se v.visitado = 0 então
        v.visitado = 1
        v.predecessor = u
        Enfileira(F, v)
```

4.2 Implementação

```
1 // Atividade 1: Implementação da Busca em Largura em C
2 // Aluno: Matheus Silva Pains
3 // Matrícula: 2024.1.0028.0032-0
4
5 // 1. Bibliotecas
6 #include <stdio.h>
7 #include <stdlib.h>
8 #include <string.h>
9
10 // 2. Definição do número máximo de vértices
11 #define MAX_V 100
12
```

```
13 // 3. Construtores e Funções de Processamento
14 typedef struct No {
15     int v;
16     struct No *prox;
17 } No;
18
19 No *adj[MAX_V];
20
21 void adicionaAresta(int u, int v) {
22     No *nu = malloc(sizeof(No));
23     nu -> v = v;
24     nu -> prox = adj[u];
25     adj[u] = nu;
26
27     No *nv = malloc(sizeof(No));
28     nv -> v = u;
29     nv -> prox = adj[v];
30     adj[v] = nv;
31 }
32
33 typedef struct {
34     int dados[MAX_V];
35     int ini, fim, tam;
36 } Fila;
37
38 void inicializaFila(Fila *f) {
39     f -> ini = f -> fim = f -> tam = 0;
40 }
41
42 int filaVazia(Fila *f) {
43     return f -> tam == 0;
44 }
45
46 void enqueue(Fila *f, int x) {
47     f -> dados[f -> fim] = x;
48     f -> fim = (f -> fim + 1) % MAX_V;
49     f -> tam++;
50 }
51
52 int dequeue(Fila *f) {
53     int x = f -> dados[f -> ini];
```

```

54     f -> ini = (f -> ini+1)%MAX_V;
55     f -> tam--;
56     return x;
57 }
58
59 // R4. Função para implementar a Busca em Largura
60 void bfs(int s) {
61     int visitado[MAX_V];
62     memset(visitado, 0, sizeof(visitado));
63
64     Fila f;
65     inicializaFila(&f);
66
67     visitado[s] = 1;
68     enfileira(&f, s);
69
70     // R5. Escreva um programa em C que mostre a ordem em que os
71     // vértices foram visitados.
72     printf("Ordem de visita (BFS a partir de %d): ", s);
73
74     while (!filaVazia(&f)) {
75         int u = desenfileira(&f);
76         printf("%d ", u);
77
78         for (No *p = adj[u]; p != NULL; p = p -> prox) {
79             int v = p -> v;
80             if (!visitado[v]) {
81                 visitado[v] = 1;
82                 enfileira(&f, v);
83             }
84         }
85     }
86     printf("\n");
87 }
88
89 // 4. Main
90 int main(void) {
91     int n, m, s;
92
93     // R1. Escrever um programa em C que leia a quantidade de vé
94     // rtices e arestas do grafo.

```

```
93     printf("Numero de vertices: ");
94     scanf("%d", &n);
95
96     printf("Numero de arestas: ");
97     scanf("%d", &m);
98
99     memset(adj, 0, sizeof(adj));
100
101     // R2. Escrever um programa em C que leia as arestas do
102         grafo.
103     printf("Arestas (u v):\n");
104     for (int i = 0; i < m; i++) {
105         int u, v;
106         scanf("%d %d", &u, &v);
107         adicionaAresta(u, v);
108     }
109
110     // R3. Escrever um programa em C que leia o vértice inicial.
111     printf("Vertice inicial: ");
112     scanf("%d", &s);
113
114     // R4. Escreva um programa em C que executa a Busca em
115         Largura a partir do vértice inicial.
116     bfs(s);
117     return 0;
118 }
```

Listing 1: bfs.c – Busca em Largura em grafo não direcionado

4.3 Resultados

A Figura 1 mostra dois testes executados com o mesmo grafo de 6 vértices e 7 arestas $\{(1, 2), (1, 3), (2, 3), (2, 4), (3, 4), (3, 5), (5, 6)\}$, variando apenas o vértice inicial.

```

PS C:\Users\matheuspains\CC_2026-1_Projects> .\bfs.exe
Numero de vertices: 6
Numero de arestas: 7
Arestas (u v):
1 2
1 3
2 3
2 4
3 4
3 5
5 6
Vertice inicial: 4
Ordem de visita (BFS a partir de 4): 4 3 2 5 1 6
PS C:\Users\matheuspains\CC_2026-1_Projects>
PS C:\Users\matheuspains\CC_2026-1_Projects> .\bfs.exe
Numero de vertices: 6
Numero de arestas: 7
Arestas (u v):
1 2
1 3
2 3
2 4
3 4
3 5
5 6
Vertice inicial: 1
Ordem de visita (BFS a partir de 1): 1 3 2 5 4 6
PS C:\Users\matheuspains\CC_2026-1_Projects>

```

Figure 1: Saída do programa BFS: vértice inicial 4 (acima) e vértice inicial 1 (abaixo).

Vértice inicial	Ordem de visita	Níveis
4	4 3 2 5 1 6	0; 1; 2; 3
1	1 3 2 5 4 6	0; 1; 2; 3

Como esperado, a ordem de visita muda com o vértice inicial, mas a lógica de exploração por níveis é mantida em ambos os casos.

5 Busca em Profundidade (DFS)

5.1 Descrição do algoritmo

A DFS explora o grafo seguindo **um ramo o mais profundamente possível** antes de retroceder (*backtrack*). O critério de escolha é sempre o vértice *mais recentemente alcançado* que ainda possui vizinhos não visitados.

A estrutura de dados correspondente é a **pilha** (LIFO — *Last In, First Out*).

O pseudocódigo seguido na implementação é o das Aulas 10 e 11:

```
BuscaProfundidade(G, s)
    s.visitado = 1
    cria pilha vazia P
    Empilha(P, s)
    enquanto P.tamanho > 0 faça
        u = Consulta(P)
        se existe uv em E(G) com v.visitado = 0 então
            v.visitado = 1
            v.predecessor = u
            Empilha(P, v)
        senão
            Desempilha(P)
```

5.2 Implementação

```
1 // Atividade 2: Implementação da Busca em Profundidade em C
2 // Aluno: Matheus Silva Pains
3 // Matrícula: 2024.1.0028.0032-0
4
5 // 1. Bibliotecas
6 #include <stdio.h>
7 #include <stdlib.h>
8 #include <string.h>
9
10 // 2. Definição do número máximo de vértices
11 #define MAX_V 100
12
13 // 3. Construtores e Funções de Processamento
14 typedef struct No {
15     int v;
16     struct No *prox;
17 } No;
18
19 No *adj[MAX_V];
20
21 void adicionaAresta(int u, int v) {
22     No *nu = malloc(sizeof(No));
23     nu -> v = v;
24     nu -> prox = adj[u];
25     adj[u] = nu;
26
```

```
27     No *nv = malloc(sizeof(No));
28     nv -> v = u;
29     nv -> prox = adj[v];
30     adj[v] = nv;
31 }
32
33 typedef struct {
34     int dados[MAX_V];
35     int topo;
36 } Pilha;
37
38 void inicializaPilha(Pilha *p) {
39     p -> topo = -1;
40 }
41
42 int pilhaVazia(Pilha *p) {
43     return p -> topo < 0;
44 }
45
46 void empilha(Pilha *p, int x) {
47     p -> dados[++(p -> topo)] = x;
48 }
49
50 int consulta(Pilha *p) {
51     return p -> dados[p -> topo];
52 }
53
54 int desempilha(Pilha *p) {
55     return p -> dados[(p -> topo)--];
56 }
57
58 // R4. Função para implementar a Busca em Profundidade
59 void dfs(int s) {
60     int visitado[MAX_V];
61     memset(visitado, 0, sizeof(visitado));
62
63     Pilha p;
64     inicializaPilha(&p);
65
66     visitado[s] = 1;
67     empilha(&p, s);
```

```
68
69 // R5. Escreva um programa em C que mostre a ordem em que os
    vértices foram visitados.
70 printf("Ordem de visita (DFS a partir de %d): ", s);
71 printf("%d ", s);
72
73 while (!pilhaVazia(&p)) {
74     int u = consulta(&p);
75     int achou = 0;
76
77     for (No *viz = adj[u]; viz != NULL; viz = viz -> prox) {
78         if (!visitado[viz -> v]) {
79             visitado[viz -> v] = 1;
80             empilha(&p, viz -> v);
81             printf("%d ", viz -> v);
82             achou = 1;
83             break;
84         }
85     }
86
87     if (!achou)
88         desempilha(&p);
89 }
90 printf("\n");
91 }
92
93 // 4. Main
94 int main(void) {
95     int n, m, s;
96
97     // R1. Escrever um programa em C que leia a quantidade de vé
        rtices e arestas do grafo.
98     printf("Numero de vertices: ");
99     scanf("%d", &n);
100
101     printf("Numero de arestas: ");
102     scanf("%d", &m);
103
104     memset(adj, 0, sizeof(adj));
105
```

```
106 // R2. Escrever um programa em C que leia as arestas do
    grafo.
107 printf("Arestas (u v):\n");
108 for (int i = 0; i < m; i++) {
109     int u, v;
110     scanf("%d %d", &u, &v);
111     adicionaAresta(u, v);
112 }
113
114 // R3. Escrever um programa em C que leia o vértice inicial.
115 printf("Vertice inicial: ");
116 scanf("%d", &s);
117
118 // R4. Escreva um programa em C que executa a Busca em
    Profundidade a partir do vértice inicial.
119 dfs(s);
120 return 0;
121 }
```

Listing 2: dfs.c – Busca em Profundidade em grafo não direcionado

5.3 Resultados

A Figura 2 mostra dois testes executados com o mesmo grafo utilizado na BFS, variando o vértice inicial.

```

PS C:\Users\matheuspains\CC_2026-1_Projects> .\dfs.exe
Numero de vertices: 6
Numero de arestas: 7
Arestas (u v):
1 2
1 3
2 3
2 4
3 4
3 5
5 6
Vertice inicial: 4
Ordem de visita (DFS a partir de 4): 4 3 5 6 2 1

PS C:\Users\matheuspains\CC_2026-1_Projects> .\dfs.exe
Numero de vertices: 6
Numero de arestas: 7
Arestas (u v):
1 2
1 3
2 3
2 4
3 4
3 5
5 6
Vertice inicial: 1
Ordem de visita (DFS a partir de 1): 1 3 5 6 2 4

PS C:\Users\matheuspains\CC_2026-1_Projects>

```

Figure 2: Saída do programa DFS: vértice inicial 4 (acima) e vértice inicial 1 (abaixo).

Vértice inicial	Ordem de visita
4	4 3 5 6 2 1
1	1 3 5 6 2 4

O comportamento de “mergulho” da DFS fica evidente: a partir do vértice inicial, o algoritmo avança por um único caminho até não conseguir mais prosseguir, e só então retrocede para tentar outro ramo.

6 Comparação entre BFS e DFS

A Tabela 1 resume as principais diferenças entre os dois algoritmos.

Table 1: Comparação entre BFS e DFS

Característica	BFS	DFS
Estrutura de controle	Fila (FIFO)	Pilha (LIFO)
Estratégia de exploração	Por níveis	Por profundidade
Critério de escolha	Mais antigo com vizinhos livres	Mais recente com vizinhos livres
Árvore gerada	Árvore de largura	Árvore de profundidade
Complexidade	$O(V + E)$	$O(V + E)$
Detecta ciclos?	Sim (aresta entre visitados)	Sim (aresta entre visitados)

Ambos os algoritmos visitam cada vértice e cada aresta no máximo uma vez, resultando na mesma complexidade assintótica $O(V + E)$. A diferença está na *ordem* em que os vértices são explorados, o que pode ser relevante dependendo do problema a resolver — BFS é preferível quando se busca o caminho mais curto, enquanto DFS é base para algoritmos como detecção de ciclos e ordenação topológica.

7 Conclusão

As implementações desenvolvidas seguem fielmente os pseudocódigos apresentados nas Aulas 10 e 11, utilizando lista de adjacência para a representação do grafo, fila circular estática para a BFS e pilha estática para a DFS. Os testes realizados confirmaram o comportamento esperado de cada algoritmo: exploração por níveis na BFS e exploração em profundidade na DFS. A variação do vértice inicial alterou a ordem de visita em ambos os casos, o que é consistente com a observação do material de referência de que a árvore de busca não é única.